

Lecture 7 : Proof by Induction Continued

September 15, 2026

Scribe: Alejandro Otermin

This lesson focused on working and clarifying the reasoning behind a proof. We hit multiple errors throughout the class and then spent the last section fixing it. That error and its fix are both documented below (in the red and green boxes).

1 Quick Terminology Recap

- **Strong induction:** to prove a statement $P(n)$ for all $n \geq n_0$: (1) prove one or more base cases directly, and (2) assume the *inductive hypothesis* — that $P(i)$ holds for *every* i with $n_0 \leq i < n$ — and use that assumption to prove $P(n)$, i.e. show

$$P(i) \Rightarrow P(i + 1).$$

(This is "strong" because you get to assume P holds for *all* smaller values, not just the one immediately before n .)

- x^n means x multiplied by itself n times; $\log$ below denotes $\log_2$ unless stated otherwise.

2 Every Integer ≥ 2 Is a Product of Primes

Claim 1. *Every integer $r \geq 2$ can be written as a product of primes:*

$$r = p_1^{a_1} p_2^{a_2} \cdots p_k^{a_k}.$$

Examples: $2 = 2$, $4 = 2 \cdot 2$, $14 = 7 \cdot 2$, $12 = 2^2 \cdot 3$.

Proof. By strong induction on r .

Base case: $r = 2$. Since 2 is itself prime, it is a product of one prime, so $P(2)$ holds.

Inductive hypothesis: for every i with $2 \leq i < n$, i can be written as a product of primes.

Inductive step: consider n . There are two cases.

- *Case 1: n is prime.* Then n is a product of a single prime (itself), so $P(n)$ holds.
- *Case 2: n is not prime.* Then by definition $n = a \times b$ for some $2 \leq a, b < n$. By the inductive hypothesis, both a and b can be written as products of primes. Writing both factorizations over the *same* list of primes $p_1, \dots, p_k$ (allowing an exponent of 0 for any prime that doesn't actually divide one of them),

$$a = p_1^{\alpha_1} \cdots p_k^{\alpha_k}, \quad b = p_1^{\beta_1} \cdots p_k^{\beta_k}.$$

Multiplying two numbers written this way just adds exponents, so

$$n = a \cdot b = p_1^{\alpha_1 + \beta_1} \cdots p_k^{\alpha_k + \beta_k},$$

which is again a product of primes. So $P(n)$ holds.

This completes the induction.

3 Main proof: How Expensive Is Multiplying Two Numbers?

Most of the day was spent proving this algorithm, which consisted of multiplying two large numbers, e.g.

$$12764 \times 38975.$$

Multiplying two n -digit numbers with the standard grade-school method costs $O(n^2)$ single-digit multiplications.

- It's possible to do this in $O(n \log n)$ time instead, but that kind of algorithm is really the subject of an algorithms course, not this one.
- In cryptography, where numbers routinely have around 200 digits, the gap between n^2 and $n \log n$ can actually start to matter in practice.

Throughout class we analyzed different algos for computing x^n efficiently.

4 Fast Exponentiation: Computing x^n

4.1 The naive approach

The obvious way to compute x^N is

$$x \cdot x \cdot x \cdots x \quad (N \text{ times}),$$

i.e. $N - 1$ multiplications. The runtime of this approach depends on N and on the cost of each individual multiplication, which itself depends on the number of digits involved (recall: multiplying two numbers with roughly d digits costs about d^2). But since the naive method does $\Theta(N)$ multiplications, the runtime is mostly dominated by N itself, however large the numbers get.

4.2 A faster, recursive algorithm

Rephrase of the idea: if the exponent is very large, break it into two smaller "chunks" (halves), solve each recursively, and multiply the results back together.

$$\text{pow}(x, n) :$$

Base case: if $n = 0$, return 1.

n even: if $n = 2k$, return $\text{pow}(x, n/2) \times \text{pow}(x, n/2)$.

n odd: if $n = 2k + 1$, return $\text{pow}(x, \lfloor n/2 \rfloor) \times \text{pow}(x, \lfloor n/2 \rfloor) \times x$.

We now want to prove two things about this algorithm: that it's **correct**, and that it's **efficient**.

4.3 Correctness

Claim 2. For all $n \geq 0$, $\text{pow}(x, n) = x^n$.

Proof. By strong induction on n .

Base case: $P(0)$: $\text{pow}(x, 0) = 1 = x^0$.

Inductive hypothesis: for all i with $0 \leq i < n$, $\text{pow}(x, i) = x^i$.

Inductive step:

- If n is even, $n = 2k$ (with $k < n$): by the IH, $\text{pow}(x, k) = x^k$, so

$$\text{pow}(x, n) = \text{pow}(x, k) \cdot \text{pow}(x, k) = x^k \cdot x^k = x^{2k} = x^n.$$

- If n is odd, $n = 2k + 1$: similarly,

$$\text{pow}(x, n) = x^k \cdot x^k \cdot x = x^{2k+1} = x^n.$$

4.4 Efficiency

Let $M(n)$ denote the number of multiplications $\text{pow}(x, n)$ performs.

Claim 3. For $n \geq 1$, $M(n) \leq 2\lceil \log n \rceil + 2$.

Base cases: $M(0) = 0$ and $M(1) = 0$ multiplications, both trivially satisfying the bound.

Inductive hypothesis: for all $1 \leq i < n$, $M(i) \leq 2\lceil \log i \rceil + 2$.

What the algorithm is actually doing: to compute $\text{pow}(x, n)$, it (conceptually) computes $y = \text{pow}(x, \lceil n/2 \rceil)$ once, using $M(\lceil n/2 \rceil)$ multiplications, and then *combines* it:

- if n is even: one more multiplication, $y \cdot y$;
- if n is odd: two more multiplications, $y \cdot y \cdot x$.

So the combining step costs *at most* 2 extra multiplications, which gives the recurrence

$$M(n) \leq M(\lceil n/2 \rceil) + 2.$$

This is where the first attempt on the board went wrong.

Mistake made in class

The board's first pass treated the two calls to $\text{pow}(x, n/2)$ in the pseudocode as two *separate, independently-costed* recursive calls, giving

$$M(n) \leq 2M(n/2) + 2.$$

Substituting the inductive hypothesis:

$$2M(n/2) + 2 \leq 2[2\log(n/2) + 2] + 2.$$

The next step then simplified $\log(n/2)$ as if it were just $\log n$, instead of $\log n - 1$. That's the

error that got flagged live: it should have been $\log(n) - 1$, not $\log(n)$. With that slip, the algebra didn't land on the target bound at all.

Correction

Two things needed fixing:

1. **The recursion itself:** $y = \text{pow}(x, \lceil n/2 \rceil)$ only needs to be accounted for *once* — the algorithm's cost is driven by one recursive subproblem of size $n/2$, plus at most 2 multiplications to combine. So the correct recurrence is

$$M(n) \leq M(n/2) + 2, \quad \text{not} \quad M(n) \leq 2M(n/2) + 2.$$

2. **The logarithm identity:** $\log(n/2) = \log n - 1$.

Correct derivation:

$$\begin{aligned} M(n) &\leq M(\lceil n/2 \rceil) + 2 \\ &\leq [2\log(n/2)] + 2 + 2 && \text{(by the inductive hypothesis)} \\ &= 2([\log n] - 1) + 4 && \text{(since } \log(n/2) = \log n - 1 \text{)} \\ &= 2[\log n] + 2 \\ &\leq 2[\log n] + 2. \end{aligned}$$

This matches the original claim, so $M(n) \leq 2[\log n] + 2$ for all $n \geq 1$.

4.5 An equivalent way to see it: binary representation

Here's another way to look at the same algorithm, using the binary representation of n .

Worked example: take $n = 13$. In binary, $13 = 8 + 4 + 1 = 2^3 + 2^2 + 2^0$, so

$$x^{13} = x^{2^3} \cdot x^{2^2} \cdot x^{2^0} = x^8 \cdot x^4 \cdot x^1.$$

In general: representing N in binary takes $\lceil \log_2 N \rceil + 1$ bits. We can build up the sequence

$$x^{2^0}, x^{2^1}, x^{2^2}, \dots, x^{2^{\lceil \log N \rceil}}$$

by repeated squaring — each term costs just one multiplication given the previous one, so this whole table takes about $\log N$ multiplications to build. This is the same idea as the recursive algorithm above, just written iteratively (build the powers of x from the bottom up instead of splitting n from the top down).

Then x^N is just the product of the entries in that table corresponding to the 1-bits of N (for the earlier example, the 2^3 , 2^2 , and 2^0 entries), which costs at most another $\log N$ multiplications to combine.

Why this matters: the naive method does N multiplications total. This method does about $\log_2 N$ multiplications total. So the total cost drops from

$$N \times (\text{cost of one multiplication}) \quad \longrightarrow \quad \log_2 N \times (\text{cost of one multiplication}),$$

which is the entire point of the algorithm.

5 Questions

1. If we applied strong induction for the prime proofs, what does weaker induction look like?

Weak induction only lets you assume $P(i)$ to prove $P(i + 1)$ — just the single previous value. But the prime proof writes $n = a \times b$ for some $a, b < n$, with no guarantee either one is $n - 1$. E.g. $12 = 3 \times 4$: weak induction only hands you $P(11)$, which tells you nothing about 3 or 4. That's why the proof needs strong induction, which assumes $P(j)$ for *every* $j < n$.

2. For the main proof, why does the correct recurrence only count $\text{pow}(x, n/2)$ once, when the pseudocode calls it twice?

The pseudocode calls $\text{pow}(x, n/2)$ twice, but both calls compute the exact same value, so in practice you'd only compute it once and reuse it (i.e. $y = \text{pow}(x, n/2)$, then return $y * y$). The recurrence counts the *work*, and there's only one distinct subproblem being solved — so it's $M(n) \leq M(n/2) + 2$, not $M(n) \leq 2M(n/2) + 2$.

3. Where did we get the $+2$ from in the recurrence relation for the Main proof?

The $+2$ covers the multiplications needed to *combine* the recursive result. If n is even, you only need one extra multiplication ($y \times y$). If n is odd, you need two ($y \times y \times x$). So $+2$ is just a safe upper bound that covers both cases at once, rather than writing the recurrence differently for even and odd n .

4. For the mistake made on the main proof, why is simplifying $\log(n/2)$ using just $\log n$ as opposed to $\log n - 1$ wrong?

Logarithms turn division into subtraction: $\log(n/2) = \log n - \log 2 = \log n - 1$ (since $\log_2 2 = 1$). So $\log(n/2)$ and $\log n$ aren't the same thing — they're off by exactly 1. Dropping that -1 makes the bound look tighter than it actually is, which is why the first attempt at the algebra didn't correctly reduce to the target bound.

5. Why does the binary-representation method (building $x^{2^0}, x^{2^1}, \dots$) end up costing about the same as the recursive pow algorithm — are they doing the same multiplications, just in a different order?

Yes — both methods build up the same set of squared powers of x ($x^{2^0}, x^{2^1}, x^{2^2}, \dots$) and then multiply together the ones that correspond to 1-bits of n . The recursive version builds these top-down (splitting n in half each time), while the

binary version builds them bottom-up (squaring repeatedly, then combining at the end). Either way it takes about $\log_2 n$ multiplications to build the powers and at most $\log_2 n$ more to combine them — the same total cost, just computed in opposite directions.